Urbanova Build Order

Every file in the proposed repo, mapped to an owner and a build step, updated for the resident-centred Urbanova focus in project_desc.md while keeping the remaining build order, dependency chain, contracts and gates the same. Each phase unblocks the next, and the four lanes in Phase 2 only run in parallel because Phase 1 proved the data exists and froze the contract.

OUR CENTRAL QUESTION (RESIDENT-CENTRED FOCUS)
How could this development change my commute and neighbourhood, and what improvements would help accommodate that growth?

111 steps | 4 owners | 6 phases | 1 corridor, 9 stops (505 Dundas WB) | 3 developments
3 views (Street 3D, Shadows, Aerial Transit) | 4 packages x 3 demand cases | Fri 22:00 -> Sun 08:00

A Owner 2
Lane A | data, demand, 505 corridor, 3D massing & shadows

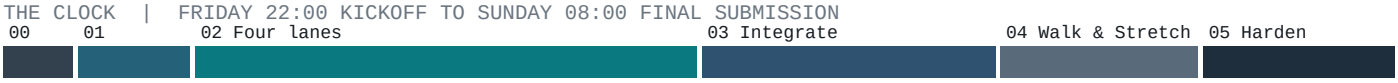
B Owner 3
Lane B | streetcar & passenger simulation, verification

C Owner 4
Lane C | Express/FastAPI, Tiger Data, Gemini/Papiers, packages

D Emilia
Lane D | 3D street/aerial views, shadows, replay, comparison, demo

Owner numbers follow section 13 of the final idea. Tick steps as they land; all remaining steps, schemas and gates stay unchanged.

THE CLOCK | FRIDAY 22:00 KICKOFF TO SUNDAY 08:00 FINAL SUBMISSION



00 Foundation

All four together | Fri 23:45 -> Sat 00:30 | 0 / 8 done

Nobody writes product code until all four laptops run the backend, the frontend and a database connection. Fixing one broken environment at hour 20 costs far more than fixing it together now.

ALL HANDS

Make it run on four laptops

- [ ] 0.1 **`requirements.txt | backend/package.json`**
The backend is Express (Node) alongside FastAPI schemas; Python powers the data pipelines (Lane A) and the simulator (Lane B). Pin Python versions: numpy, pandas, geopandas, shapely, pyproj, osmnx, psycopg[binary], pyyaml, python-dotenv, pytest. Pin backend npm versions: express, cors, pg, zod, @google/genai, dotenv. Pinning now prevents the 'works on mine' hour.

- [ ] 0.2 **`.gitignore`**
Add node_modules/, __pycache__/, .env, data/cache/*, data/raw/*, dist/ before the first npm install. The city massing download must never reach git.

- [ ] 0.3 **`.env.example`**
GEMINI_API_KEY, GEMINI_MODEL, DATABASE_URL (Tiger Data), DATABASE_URL_LOCAL (fallback), DATA_CACHE_DIR, SIM_RUNS, OFFLINE. Keys stay on the backend.

- [ ] 0.4 **backend/config.js**
  One zod-validated config object loading .env, plus MODEL_VERSION and DATA_VERSION strings that every saved result records.

- [ ] 0.5 **docker-compose.yml**
  Local PostgreSQL + PostGIS with the same schema as Tiger Data. If venue wifi drops, the demo switches database URL and keeps working.

- [ ] 0.6 **frontend/**
  npm create vite@latest frontend -- --template react-ts, then add maplibre-gl and one chart library. Owner 1 decides which and nobody adds a second.

- [ ] 0.7 **scripts/setup.sh**
  venv -> pip install -r requirements.txt -> npm install in backend/ and frontend/ -> database ping. One command per teammate.

- [ ] 0.8 **README.md**
  Ten lines: what Urbanova is (helping Toronto residents picture how proposed housing affects the buildings they see, the sunlight in places they use, and the streetcar transit they depend on), and how to run the backend, worker and frontend. Expand at the end.

GATE - DO NOT PROCEED UNTIL

All four people have the Express backend (npm run dev in backend/), the frontend (npm run dev in frontend/) and SELECT postgis_version(); working locally, and a first commit is pushed.

01 Prove feasibility, freeze the contract

All four together | Sat 00:30 -> 02:30 | blocking | 15 / 16 done

UPDATED FOCUS & STATUS: Study area is TTC 505 Dundas westbound, Broadview -> TMU Station at Yonge/Dundas (9 stops) with three selected proposals: 239 Dundas St E, 500 Dundas St E and 761 Dundas St E. Existing residents are the primary audience exploring three connected views: Street-Level 3D View, Sunlight & Shadows, and Aerial View & Streetcar Transit. See simulation-data.md, massing.md and contract.md (Contract 2.0.0) for executed checks.

Choose the corridor only after inspecting real files, and agree IDs, units, direction, coordinates, scenario fields, event types and result fields before splitting. The simulation and data owners work side by side: a demand and model mismatch can invalidate two working lanes at once.

IN PAIRS | TIME-BOXED TO 75 MINUTES

Feasibility checks

- [x] 1.1 **docs/corridor.md** DECISION
  Resolved to TTC 505 Dundas streetcar, westbound, Broadview Ave -> TMU Station at Yonge/Dundas (9 consecutive stops, 8 links, 07:00-09:00 AM peak window). In the AM peak, new residents near River, Regent Park and Parliament board westbound toward downtown and Line 1, so crowding they add reaches existing riders further along.

Selected: TTC 505 Dundas Westbound

Broadview Ave (1756) -> TMU Station at Yonge/Dundas (2136). 9 consecutive stops, 8 links (2.51 km), 07:00-09:00 AM peak (17 scheduled streetcars, 6 min median headway). Three selected proposals: 239 Dundas St E (624 net units), 500 Dundas St E (413 units) and 761 Dundas St E (1,197 units).

Historical Bus Candidates (Eglinton E / Lawrence E)

Retained from initial planning: Eglinton East-Kingston Rd-Morningside (Routes 86/116/905/986, RapidTO lanes) and Lawrence Ave East (Routes 54/154/954, unfunded RapidTO). Replaced by 505 Dundas westbound for the connected resident 3D, seasonal shadow and streetcar commute workflow.

- [x] 1.2 **`docs/feasibility.md`**
  Confirm three developments exist in the Development Pipeline with unit counts and stages (239 Dundas St E: 624 net / 630 proposed; 500 Dundas St E: 413 net; 761 Dundas St E: 1,197 provisional net), and that the GTFS feed has the chosen 505 westbound pattern on a weekday service date. Because stop-level demand observations stay unavailable, the product is labelled an exploratory scenario experiment.

- [x] 1.3 **`backend/scripts/check_access.py`**
  Tiger Data service reachable with PostGIS enabled; one Gemini call returns a valid structured response. If either fails, decide the fallback now, not at hour 20.

- [ ] 1.4 **`Simulation owner's hand-case validation`** OWNER 3'S FIRST MILESTONE
  Three stops, two streetcars, ten passengers, worked out by hand first and written in the docstring. The simulator's first job is to reproduce that page exactly (no duplicate root sim/ implementation).

ALL HANDS | STRICT ORDER

Schemas, innermost to outermost

Each object below is composed by the next. Schemas are Pydantic modules in backend/schemas/*.py (reusing Stop and Development dataclasses from backend/simulation/objects/); Python pipelines, Express and the simulator share identical field names.

- [x] 1.5 **`backend/schemas/source.py`**
  SourceRef: source_id, title, url, publisher, retrieved_at, version, licence. Every other object points at these ids.

- [x] 1.6 **`backend/schemas/assumption.py`** CRITICAL
  Assumption: key, unit, meaning, low / central / high, allowed range, status (sourced | assumed | user_selected), source_ids. The UI shows unit, meaning, status and range on every control, and this is where they come from.

- [x] 1.7 **`backend/schemas/corridor.py`**
  Stop (id, name, sequence, latitude, longitude, waiting_passenger_ids, serving_streetcar_id), Link (link_id, from_stop, to_stop, length_m, base_time_s), Corridor (route pattern, direction, peak window, stops[], links[], berths per stop, incoming load at the first stop).

- [x] 1.8 **`backend/schemas/development.py`**
  Development: id, name, latitude, longitude, net_added_units, occupied_share, residents_per_occupied_unit, transit_share, accessible_stop_ids (plus sourced pipeline metadata in data/reviewed). Units are proposed; residents exist only after an occupancy assumption.

- [x] 1.9 **`backend/schemas/intervention.py`**
  Intervention: id, type (priority_lane | extra_service), link_ids or headway, capital_cost and operating_cost ranges with price year and scope, extra vehicles, earliest_phase, effect with status.

- [x] 1.10 **backend/schemas/scenario.py**
  Scenario: scenario_id, version, developments per phase, occupied share, demand case, intervention availability dates, capital and operating budgets, fleet allowance, planning criterion. Housing totals are fixed across package comparisons.

- [x] 1.11 **backend/schemas/run.py**
  Run: run_id, scenario_version, package, model_version, data_version, seed ids, status, diagnostics (conservation, unresolved passengers, runtime).

- [x] 1.12 **backend/schemas/replay_event.py**
  ReplayEvent: run_id, t_s, kind (dispatch | arrive | alight | board | deny | depart), vehicle_id, stop_id, count, load_after, rider_group. This one type drives the moving streetcars, the stop queues and the replay stored in Tiger Data.

- [x] 1.13 **backend/schemas/results.py**
  StopResult by rider group (existing, new): mean and p95 wait, share denied at least once, remaining queue, headway variability. CaseResult adds per-run arrays, run count and unresolved passengers.

- [x] 1.14 **backend/schemas/comparison.py** CONVERGENCE
  Comparison: scenario, corridor, cases (current, growth, supported growth), packages[] with feasibility reasons and criterion results per phase and demand case, sensitivity, assumptions[], sources[], warnings[]. If a lane's output does not fit in here, that lane is building the wrong thing.

- [x] 1.15 **frontend/src/lib/types.ts**
  Hand-mirror 1.5 through 1.14 in TypeScript, with field names identical to the Pydantic models.

- [x] 1.16 **data/fixtures/sample_comparison.json** DO NOT SKIP
  One fully populated Comparison plus a short replay stream (two streetcars, every stop along westbound 505 Dundas), written by hand. Set 'fixture': true and show a FIXTURE banner whenever it is on, so made-up numbers never reach a judge.

GATE - DO NOT PROCEED UNTIL

The 505 Dundas westbound corridor is chosen from real files, the tiny case matches the hand calculation, sample_comparison.json validates against the Pydantic models and the TypeScript types compile against it. Then split up.

02 Four lanes in parallel

Sat 02:30 -> 13:00 | the long stretch | sleep in staggered pairs | 0 / 56 done

Each lane runs and tests on its own. Order matters within a lane, not across lanes. Two cross-lane waits, both stubbed until they land: B3 needs A7's walk table, and C5 needs B11's paired runner.

LANE A | OWNER 2

Corridor bundle, developments, 3D massing, shadows & demand inputs

Own what the model is allowed to believe. Every row carries a source id and retrieval date; anything you cannot source enters as an assumption with a status.

- [ ] A1 **data/sources/base.py**
  Download-once helper: on-disk cache, retrieved_at stamp, checksum. Every other source uses it.

- [ ] A2 **data/sources/gtfs_static.py**
  TTC Routes and Schedules: stops, trips, stop times and shape for the chosen 505 westbound pattern and service date. Resolve direction, branches, calendar exceptions and times after 24:00. Keep the feed's dates in its SourceRef.

- [ ] A3 **data/reviewed/developments.csv** CRITICAL
  Shortlist five Development Pipeline projects, keep the three best documented on 505 Dundas (239 Dundas St E, 500 Dundas St E, 761 Dundas St E) plus the expanded mapped-building residential inventory (building_unit_inventory.json, existing_residences.csv). Check that related applications are not counted twice. Occupancy dates and completion shares are scenario inputs unless sourced.

- [ ] A4 **backend/db/schema.sql**
  Tables for sources, developments, stops, links, walking access, interventions, scenario versions, runs, results and replay events. PostGIS geometry where spatial; enable only extensions you use. Same file on Tiger Data and the local fallback.

- [ ] A5 **data/pipelines/build_corridor.py**
  Clip the 9 westbound 505 stops (Broadview -> TMU Station), build 8 links from the shape, compute lengths in metres (UTM 17N, EPSG:26917) and store display geometry in EPSG:4326. Assert stop order and direction match GTFS.

- [ ] A6 **data/pipelines/massing.py | data/sources/shadows.py**
  3D Massing clipped around the 505 corridor (buildings_3d.geojson: 2,444 existing buildings plus 4 translucent proposal tower masses with on_proposal_site and 'simplified, not an approved design' labels), NOAA sun positions (sun_positions.json), and flat-ground shadows (shadows.geojson with today, added and removed layers across equinoxes/solstices). Never ship the city-wide file to the browser.

- [ ] A7 **data/pipelines/walk_access.py** UNBLOCKS B3
  OSMnx walk network for the supported area, cached with attribution. Walking distance (<=500 m) from each development and included residential building to up to 2 nearest usable westbound 505 stops. Retain manual_review=pending where entrance/crossing field review is unfinished.

- [ ] A8 **data/reviewed/assumptions.yaml** CRITICAL
  The demand chain, one factor per line with its denominator: occupied share -> people per unit -> peak trips -> transit share / reachability -> departure profile -> destinations. Never multiply a per-person trip rate by a separate traveller fraction that counts the same thing twice.

- [ ] A9 **data/reviewed/background.yaml**
  Existing riders by stop (from the expanded mapped-building residential inventory) and incoming load on board at Broadview Ave (12-24 riders). TTC route-level boardings (32,100 weekday boardings on 505) provide coarse context; never claim daily route totals validate stop-level peak demand.

- [ ] A10 **data/reviewed/interventions.csv**
  Two options. Priority lane: link_ids, effect range from RapidTO material or a labelled streetcar assumption, capital cost with price year, earliest phase. Extra service: new headway, round-trip cycle with layover, cost per vehicle-hour, service days. A similar project's budget is not a quote; mark it as such.

- [ ] A11 **scripts/record_vehicles.py** OPTIONAL
  Check the TTC real-time feed's access terms and fields. If it works, record corridor streetcar positions for a held-out headway check. It is weekend service, not the weekday peak, and occupancy fields may be empty.

- [ ] A12 **`scripts/seed_data.py`** BY SAT 06:00
  One run loads the whole bundle into Tiger Data and the local cache. From here the app never calls an upstream site.

- [ ] A13 **`tests/data/test_bundle.py`**
  Every row has source ids, every assumption has a status and range, units are positive, walk distances are finite and plausible.

LANE B | OWNER 3

Passenger and streetcar simulation, verification

Write the invariants before the clever parts. A simulator that loses passengers still draws a convincing animation. Bunching has to come out of the events, never out of a rule that forces it.

- [ ] B1 **`sim/demand/journeys.py`**
  Generate destination-consistent journeys (origin stop, destination stop or 'beyond segment', arrival time) from the A8 chain, each tagged existing or new. Never invent boardings and alightings separately.

- [ ] B2 **`sim/demand/streams.py`**
  Separate random streams for arrivals, link noise and a shared corridor delay, pre-drawn per run so a change in event order cannot shift later draws. The same draws feed every package.

- [ ] B3 **`sim/demand/stop_assignment.py`** WAITS ON A7
  New and existing residents to stops by walking distance. Use straight-line distance as a labelled stub until A7 lands.

- [ ] B4 **`sim/engine/events.py | state.py`**
  Event heap with stable tie-breaking; state for streetcars, passenger loads inside individual vehicles and FIFO stop queues.

- [ ] B5 **`sim/engine/stop.py`**
  Berth rule (one streetcar serves a stop at a time unless the corridor says otherwise), no-overtaking rule, alight then board up to capacity, the rest stay queued.

- [ ] B6 **`sim/engine/dwell.py`**
  An explicit dwell rule from door time, boardings and alightings. This feedback makes a late streetcar later and lets the next one catch up.

- [ ] B7 **`sim/engine/links.py`**
  Link time = base + shared corridor delay + link noise. Transit priority changes only the parameters of its listed links.

- [ ] B8 **`sim/engine/dispatch.py`**
  Arrivals at the first modelled stop (Broadview) come from the GTFS schedule with their incoming load. Extra service is an alternative timetable. Faster link travel does not add trips by itself.

- [ ] B9 **`sim/engine/simulator.py`**
  Warm-up before the window, a measured cohort defined by arrival time, and enough later service to observe its waits. Anyone still waiting at the analysis cap is reported as unresolved, with a lower-bound wait.

- [ ] B10 **`tests/sim/test_invariants.py`** BEFORE B11
  Every passenger is waiting, on board, arrived or beyond the segment. Load stays between zero and capacity. Denied riders persist and can board later, and are counted once. Event times ordered, travel times non-negative. A light regular case matches the hand result. An overloaded case grows a queue. Same inputs and seeds reproduce the stored result.

- [ ] B11 **`sim/runner/paired_runs.py`**
  Start at 20 paired runs per case. Time the full load early: 20 runs x 4 packages x 3 demand cases x 2 phases is 480 runs before baselines and delay cases. Show run count and run-to-run variation.

- [ ] B12 **`sim/metrics/`**
  Mean and p95 wait, share denied at least once, remaining queue, in-vehicle crowding, journey time and headway variability, by stop and rider group. A passenger wait percentile is not a confidence interval; label it that way.

- [ ] B13 **`sim/calibration/consistency.py`**
  Baseline run times against the schedule, totals against route boardings, and A11's recorded headways as held-out data if it exists. Report the residuals. Never tune the growth case.

- [ ] B14 **`notebooks/`**
  Scratch space. Promote stable code into the modules above; the notebook never becomes the system.

LANE C | OWNER 4

Analysis API, storage, packages, grounded AI

The model computes; Gemini translates and explains. Every number on screen comes from Lane B's runs or Lane A's sources.

- [ ] C1 **`backend/server.js | backend/api/deps.js`**
  Express app, cors for the Vite port (5173), /health that also pings the database.

- [ ] C2 **`backend/services/validate.js`**
  Every site, option, value and constraint checked against the reviewed bundle before analysis. Errors say what is wrong and the allowed range.

- [ ] C3 **`backend/packages/feasibility.js`**
  Four packages: existing, priority lane, extra service, both. Vehicles = ceil(cycle / headway). Operating cost = extra vehicle-hours x service days x cost per hour. Reject anything over capital, operating or fleet limits, or unavailable in the phase, and keep the reason.

- [ ] C4 **`backend/worker/ | jobs table`**
  A separate Node worker process spawns the Python simulator (python -m sim.runner.paired_runs) so web requests never block. A job table and polling are enough; add streaming only if the workload needs it.

- [ ] C5 **`backend/packages/evaluate.js`** WAITS ON B11
  Run every feasible package on the same demand cases and draws. Until B11 lands, a stub returns fixture results.

- [ ] C6 **`backend/packages/compare.js`**
  Mark dominated options. Name the least-cost tested package that meets the user's criterion across the selected cases, or show the alternatives when none does. Wording: 'best among the tested packages under these assumptions.'

- [ ] C7 **backend/packages/sensitivity.js**
  Low, central and high demand plus one upgrade-delay case. Report whether the preferred package changes and which stops are repeatedly worst.

- [ ] C8 **backend/db/queries.sql** TIGER DATA STORY
  The three queries to show judges: development access joined to affected stops, worst stops for a phase, and one package compared across runs and assumption versions. Store aggregates for all runs and replay events for selected runs only. Measure before claiming any speed-up.

- [ ] C9 **backend/api/routes/*.js**
  corridor | scenarios | jobs | results | replay, each returning only the frozen schemas, with source and scenario ids on every result.

- [ ] C10 **backend/core/cache.js**
  Key on the full validated input, data version, model version and random configuration. Editing an input marks earlier results stale.

- [ ] C11 **backend/ai/gemini_client.js**
  One model (@google/genai), structured output, key on the server, timeout, every call logged. If Gemini fails, the forms still work.

- [ ] C12 **backend/ai/scenario_parser.js**
  Resident or planner questions ('Keep the same homes, bring the first phase forward, compare the affordable options') become a scenario patch using existing ids only, then go through C2. Unsupported questions and missing observations get a plain answer saying so.

- [ ] C13 **backend/ai/explainer.js** DO NOT FAKE
  Explains a stored Comparison from its JSON alone to help residents understand commute and neighbourhood tradeoffs. After generation, check that every number in the text appears in the payload and every source id exists; otherwise fall back to a templated explanation.

- [ ] C14 **backend/services/brief.js**
  Readable comparison brief for community discussions and transit advocacy: compared options, costs, service outcomes, uncertainty, limitations, source versions, model version and seed ids.

- [ ] C15 **backend/tests/ai/parser_cases.test.js**
  Ten fixed questions re-run after every prompt change: valid parse, correct ids, agreement with numeric results.

LANE D | EMILIA

3D neighbourhood, sunlight & shadows, streetcar replay, comparison, accessibility

Build the whole interface against sample_comparison.json, buildings_3d.geojson, sun_positions.json and shadows.geojson from the first hour. Connect the three resident views (Street-Level View, Sunlight & Shadows, and Aerial View & Transit) around the same scenario.

- [ ] D1 **frontend/src/lib/api.ts**
  Fixture first, real base URL behind VITE_USE_FIXTURE. Flipping it in Phase 3 is the integration.

- [ ] D2 **frontend/src/styles/tokens.css**
  Type scale, palette, phase colours and status styles that use shape or text as well as colour. Set once.

- [ ] D3 **src/map/CorridorMap.tsx | MassingLayer.ts | ShadowLayer.ts**
  MapLibre aerial and street-level 3D neighbourhood view along 505 Dundas. Existing buildings render as quiet extrusions; proposed towers (239, 500, 761 Dundas St E) appear as distinguishable, translucent building masses that hide underlying on_proposal_site structures when toggled. Include date and time controls to compare ground shadows (today, added lost sunlight, and removed gained sunlight from shadows.geojson) without claiming thermal-comfort modelling.

- [ ] D4 **src/panels/DevelopmentPanel.tsx**
  Proposed and net-added units, storeys, planning status, location, shadow lengths at solar noon, source link and the date of the source record.

- [ ] D5 **src/panels/StopPanel.tsx**
  Inspect a usual stop along 505 Dundas: background demand, added development demand, streetcar service and commute results (queues, vehicle crowding, boarding chances, waiting and journey times) for the selected stop.

- [ ] D6 **src/views/ListView.tsx**
  The 2D list alternative with the same essential information as the 3D map. Build it early; it is also the fallback if WebGL misbehaves on the demo laptop.

- [ ] D7 **src/panels/ScenarioEditor.tsx**
  Developments per phase, occupied share, demand factors, headway/reliability adjustments, availability dates, budgets. Each control shows unit, meaning, status and range. The planning-phase timeline, sunlight date/time selector, and simulation clock are separate controls.

- [ ] D8 **src/replay/** `DEMO SPINE`
  Streetcars moving and queues changing along 505 Dundas, from replay events. Inspect an individual streetcar (capacity, passenger load aboard, recent boarding and alighting, times) or a stop (waiting, denied). Pause, speed, scrub. Counts on screen match the model. Labelled 'one illustrative run'.

- [ ] D9 **src/panels/CaseCompare.tsx**
  Current baseline, growth with unchanged service, and supported growth side by side: mean and p95 wait, share denied, remaining queue, journey time and headway variability, split by existing and new riders, with capital cost, operating cost and extra streetcars beside them.

- [ ] D10 **src/panels/PackageTable.tsx**
  Four packages with feasibility reasons, and whether each meets the user's criterion in each phase and demand case. A failed target is shown, never hidden.

- [ ] D11 **src/panels/Sensitivity.tsx**
  Low, central, high demand and the delay case: does the preferred package change, and which stops along 505 Dundas are worst every time.

- [ ] D12 **src/panels/Evidence.tsx** `DIFFERENTIATOR`
  Every number links to the inputs behind it: the assumption with its status, and a source link that opens the real City or TTC page.

- [ ] D13 **src/panels/AskBox.tsx**
  Resident/planner Gemini question ('Why does my stop get worse even though the new building is farther up the route?') -> interpreted fields shown and editable -> Run. Nothing changes without the user seeing what changed. Stale results are marked.

- [ ] D14 **accessibility pass**
  Keyboard for every control, visible focus, text alternatives for 3D/shadow views and charts, a pause for motion, advanced assumptions in an expandable panel.

GATE - DO NOT PROCEED UNTIL

Each lane works alone: the 505 Dundas bundle (with 3D massing and shadows) is in Tiger Data and local cache; the simulator passes B10 and returns paired baseline results; packages are ranked against the stub; the interface renders the 3D proposals, shadow comparisons and fixture replay with moving streetcars and queues. Four working parts, not yet connected.

03 Integration

All four, one table | Sat 13:00 -> 21:00 | Devpost initial due 21:00 | 0 / 7 done

Integrate before polishing. Wire the request flow in one direction and add routes only as the code beneath them works: load corridor -> edit or parse scenario -> validate -> job -> simulate feasible packages -> check invariants -> aggregate -> store -> render -> explain and export.

SEQUENCED | NOT PARALLEL

Connect the chain

- [ ] 3.1 **backend/services/pipeline.js**
  The whole chain as one function. Run node backend/services/pipeline.js --scenario demo headlessly before any HTTP is involved.

- [ ] 3.2 **routes in order**
  corridor -> scenarios -> jobs -> results -> replay. Each returns real data before the next is written.

- [ ] 3.3 **backend/core/cache.js** DEMO INSURANCE
  Precompute what the judge can reach: 2 phases x 3 demand cases x on-time or delayed upgrade, all four packages each. The live challenge then returns a real stored result, shown as a cache hit.

- [ ] 3.4 **frontend/src/lib/api.ts**
  Turn VITE_USE_FIXTURE off. If Phase 1 held, nothing else in the frontend changes.

- [ ] 3.5 **backend/tests/pipeline.test.js**
  Demo scenario end to end: invariants pass in every case, the fixture flag is off, every assumption in the response has a status, an over-budget package is rejected. Run before every commit from here.

- [ ] 3.6 **reload test**
  On the deployed flow: reload, edit an input, confirm old results go stale and new ones replace them.

- [ ] 3.7 **Devpost initial submission** SAT 21:00
  Required for prize eligibility: name (Urbanova), thirty-second resident-centred pitch, School of Cities as lead, team, repo link, one screenshot. It stays editable until the final deadline.

GATE - DO NOT PROCEED UNTIL

A resident or planner can load the 505 Dundas neighbourhood, toggle proposals in 3D, compare seasonal shadows, change a housing phase, compare resource-feasible transit packages, inspect consistent stop-level commute results and uncertainty, and export a reproducible comparison brief. An infeasible budget and an unresolved queue are reported correctly. This is the MVP. Tag the commit.

04 Walk mode, then stretch

Whoever is free | conditional, skip entirely if Phase 3 is not green | Sat 21:00 -> Sun 02:00 | 0 / 15 done

If behind, cut stretch models, extra map layers and elaborate animation first. Each item below builds out the connected Street-Level View and AI Research depth once the core loop is working.

LANE D | EMILIA | FIRST STRETCH, ~ 4-6 HOURS

Walk mode: the neighbourhood and corridor at street level

A navigable ground-level 3D view of the same scene the aerial map already draws: same existing buildings, same translucent proposed towers (239, 500, 761 Dundas St E), same shadows, same streetcar replay. Built on the MapLibre camera first so every layer is reused at no extra cost.

- [ ] 4.1 **src/map/WalkMode.ts** DEMO MOMENT
  A 'Walk the street' button drops the MapLibre camera to eye height (pitch near 85 deg) near a proposal, park or chosen 505 stop. W/S move along the street, A/D strafe, mouse drag turns. Movement is clamped to the neighbourhood and the camera never goes inside a building footprint. Esc returns to aerial mode.

- [ ] 4.2 **src/map/WalkMode.ts | PhaseToggle**
  Toggle Today / Phase 1 / Phase 2 to raise or remove the translucent proposed masses and update ground shadows. Label phases, not calendar years: the brief treats occupancy dates as scenario inputs. Proposed buildings keep the 'simplified, not an approved design' styling from D3.

- [ ] 4.3 **src/map/WalkMode.ts | stop scene**
  Stand at a 505 streetcar stop while the replay runs: streetcars pull in, onboard load updates, and the queue marker grows, from the same events and counts as aerial mode. Toggle the growth phase and watch people get left behind. This connects street-level scale directly to the commute simulation.

- [ ] 4.4 **src/panels/DevelopmentPanel.tsx**
  'See this spot today' link that opens Google Maps Street View at the site's coordinates in a new tab. Real photos for reference with no API key and no licensing risk.

SECOND PERSON ON LANE D | HIGH RISK | ~ 12-20 HOURS

Full street world and future photos

A free-roam 3D street you can walk anywhere along the 505 corridor, plus photo views showing the proposed buildings in place. It never blocks the core loop: if it is not smooth by Sun 02:00, the demo uses walk mode lite (4.1-4.4).

- [ ] 4.5 **data/pipelines/street_geometry.py**
  OSM roads, sidewalks and crossings for about 1 km around the 505 Dundas corridor, as flat polygons with widths from tags or stated defaults. Exported in local metres around a fixed corridor origin, the same origin the buildings use.

- [ ] 4.6 **src/street/StreetScene.ts**
  Three.js scene with a first-person camera: pointer-lock mouse look, WASD, eye height, collision against building footprints. Loads the 4.5 geometry and the same massing GeoJSON as the map. Enter from the map's 'Walk the street' button at the camera's location.

- [ ] 4.7 **src/street/Buildings.ts**
  Extruded massing with a procedural window-grid facade scaled to height, so it reads as a street without real textures. Proposed buildings use a distinct translucent material and follow the shared phase toggle (Today / Phase 1 / Phase 2).

- [ ] 4.8 **src/street/Transit.ts**
  Streetcars and stop queues driven by the same replay events as map mode, so the street view and the charts can never disagree.

- [ ] 4.9 **data/photos/ | src/street/FuturePhoto.tsx** NOT GOOGLE
  Before/after slider at 3-5 points on the corridor. Use photos the team takes on the street or Mapillary images under their licence with attribution, not Google Street View. Render the proposed massing from a camera matched to each photo and overlay it. Every image is stamped 'Illustration - not an approved design'.

- [ ] 4.10 **performance check | Sun 02:00**
  Smooth on the demo laptop with the replay running. If not, ship walk mode lite and show the photo slider only. Decide at 02:00, not at the demo.

THEN PICK FROM THE TOP | STOP WHEN TIME RUNS OUT

Depth that survives questions

- [ ] 4.11 **deploy/vultr/** C
  Host the API and worker on Vultr if you have access, and show a scenario computed there. No GPU needed.

- [ ] 4.12 **sim/engine/signals.py** B
  Signal priority with signal delay separated from other link delay, so lane and signal benefits are not counted twice.

- [ ] 4.13 **sim/engine/holding.py** B
  A validated headway-management option, labelled a model scenario, not TTC practice.

- [ ] 4.14 **data/reviewed/hypothetical_sites.csv** A | D
  Same-total housing alternatives on a few reviewed hypothetical sites. Moving a building on screen does not make the site developable; say so in the UI.

- [ ] 4.15 **backend/research/investigator.js** PAPIERS | OPTIONAL
  Bottleneck investigator for resident and planner questions ('Why does my stop get worse even though the new building is farther up the route?'): hypothesis -> controlled experiment on matched seeds -> evaluation -> next hypothesis, with Papiers read and write. Minimum: one full investigation with an agent-chosen next step and human redirection. Findings describe the model, not the TTC.

05 Demo hardening

All four | Sun 02:00 -> 08:00 | non-negotiable | 0 / 9 done

Four minutes to present. A rehearsed demo of 80% of the system beats an unrehearsed demo of all of it. The doc lists conflicting event times, so check the organizers' current notices tonight and move these times if needed.

ALL HANDS

Make it survive contact with judges

- [ ] 5.1 **scripts/run_demo.sh**
  One command: check Tiger Data (fall back to local), confirm the cached grid, start API, worker and frontend, open the 505 Dundas neighbourhood and corridor. Run it from a cold boot at least once.

- [ ] 5.2 **docs/demo_script.md**
  Follows the six-step Resident Experience from project_desc.md with names against each segment. 0:00-0:40 Find a familiar place & see the change (505 Dundas, toggle 239/500/761 Dundas St E in 3D from street level and above) | 0:40-1:15 Explore sunlight (compare today, added and removed shadows by date/time) | 1:15-2:15 Inspect the commute & test a response (queues, streetcar crowding, waits; compare transit packages with housing held constant) | 2:15-3:15 AI research & sensitivity (Papiers/Gemini bottleneck investigation, delayed upgrade or higher demand) | 3:15-4:00 Use the evidence (source chain and saved brief). Rehearse three times.

- [ ] 5.3 **docs/qa_card.md**
  Answers for passenger conservation, incoming loads at Broadview, how shadow geometry and priority effects were chosen, why extra service costs vehicles, and why the result differs from an average-capacity sum. Keep the hand-worked tiny case from 1.4 on it.

- [ ] 5.4 **usability check**
  Someone outside the team explores a proposal's 3D scale, sunlight change and commute impact, finds which tested transit package supports the selected phase within budget, and explains why. Record completion, time and any misread uncertainty.

- [ ] 5.5 **OFFLINE=1**
  Everything served from cache and the local database with Gemini off. Test with wifi physically off.

- [ ] 5.6 **recorded backup**
  Screen recording of the full demo path after the second rehearsal, labelled as recorded.

- [ ] 5.7 **README.md | docs/assumptions.md | docs/architecture.md**
  Screenshots, the request-flow diagram, the assumptions table with statuses, and the limits: no confirmed stop-level demand data, schedules are not observed headways, flat-ground shadows do not model temperature or thermal comfort, and transport outcomes are not emission claims.

- [ ] 5.8 **feature freeze** SUN 06:00
  Bug fixes only. Everyone commits and pulls; one laptop becomes the demo laptop and stops changing.

- [ ] 5.9 **Devpost final** SUN 08:00
  Submit as Urbanova. Lead with School of Cities; Climate Action as the theme; AI for Social Impact if the Gemini/Papiers interaction works. Select Gemini, Tiger Data or Vultr only for integrations you actually demonstrate, and confirm each category's eligibility.

* Standing rules

True in every phase

Eight rules that hold from hour zero. Most exist because breaking them is silent: you find out during the demo, or a judge finds out for you.

Cache to disk by Sat 06:00

GTFS, the 3D massing clip, solar positions and shadow layers, the walk network, development records and any recordings. The demo never waits on venue wifi.

Every number & geometry has a status

Sourced, assumed or user-selected, visible next to the value. Distinguish proposed designs, simplified 3D masses and hypothetical changes; compare added/removed shadows with today at the same date and time.

The model computes, Gemini translates

Gemini fills scenario fields from existing ids and explains stored results. It never produces passenger counts, costs, shadow geometry, effects or permissions.

Hold housing constant

Packages are compared on the same homes and demand. A failed target never quietly removes homes or demand.

Matched draws for every case

Same seeds and streams across baseline, growth and every package, or the difference you show is noise.

Show the bad mornings & both sides

p95 wait, in-vehicle crowding, share left behind and unresolved passengers sit beside every mean; additional homes, seasonal shade and transit improvements sit alongside pressures.

Frontend never waits on backend

The fixture from 1.16 keeps Lane D at full speed. If the interface is blocked on an endpoint, Phase 1 went wrong.

Report what the model returns

'Best among the tested packages under these assumptions.' If the preferred package never changes, say so. No claims about thermal comfort, emissions, real waits or corridor collapse.

Updated for Urbanova from docs/doc/project_desc.md and the final idea tab of the brainstorming doc, mapped onto the repository layout. Event times come from the Hack the Valley info section; confirm them against current organizer notices. Reference sources checked 16-17 Oct 2026:

- VU.CITY: 3D urban planning and shadow analysis (https://www.vu.city/planners)
- ArcGIS Urban: 3D scenario modelling (https://learn.arcgis.com/en/projects/explore-an-urban-model/)
- TTC: 505 Dundas route and TMU Station (https://www.ttc.ca/routes-and-schedules/505/1/2136)
- TTC: 2025 operating statistics, boardings by corridor (32,100 weekday 505 boardings)
- Toronto Open Data: Merged GTFS TTC Routes and Schedules & 3D Massing
- TTC & City of Toronto: RapidTO bus/streetcar priority reports & Steve Munro travel-time analyses